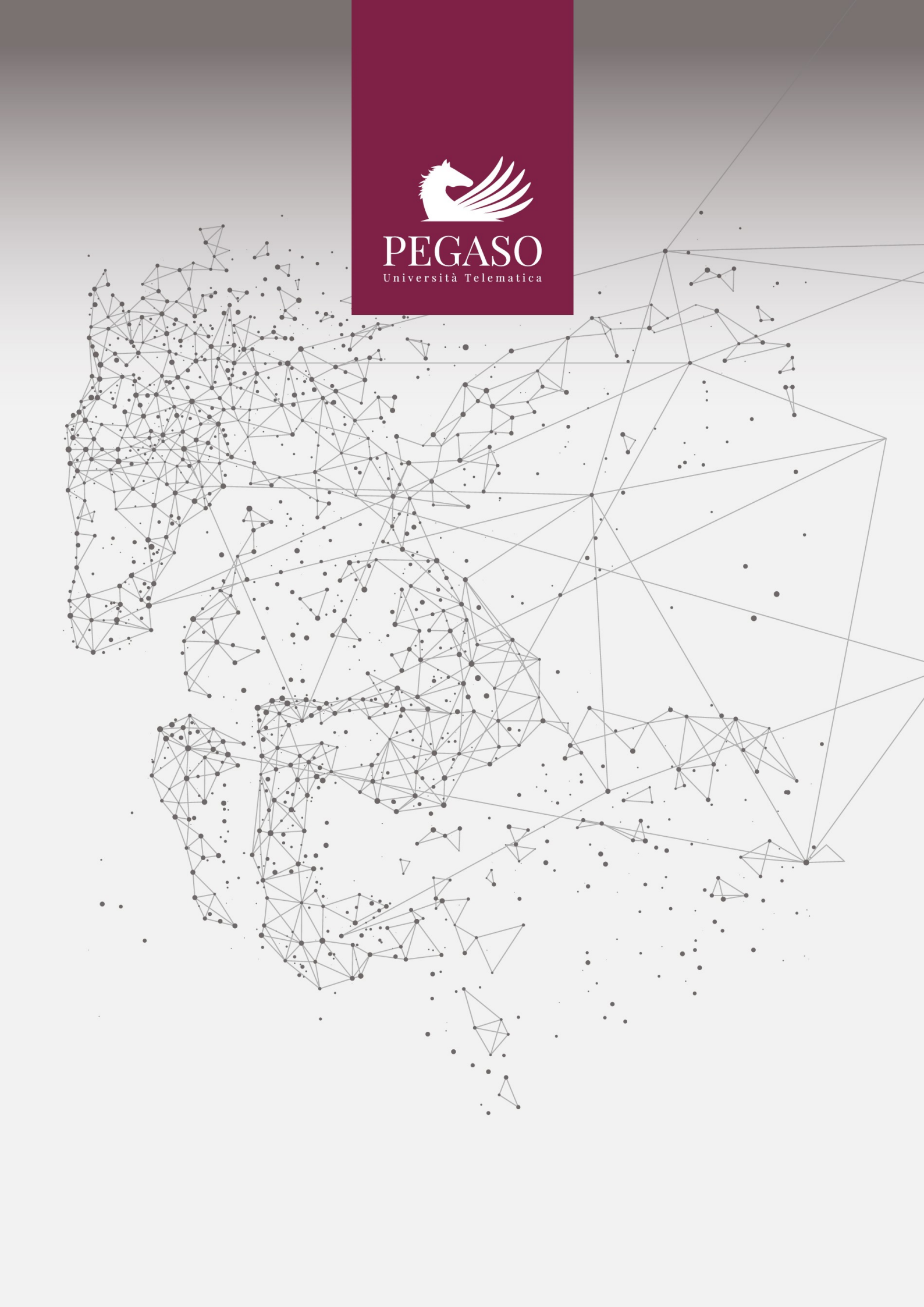

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE.....3
2. ARCHITETTURE CLIENT-SERVER 4
3. ARCHITETTURE MULTILIVELLO 6
BIBLIOGRAFIA 9

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

Lo scopo di questa lezione è esaminare come i sistemi distribuiti sono effettivamente organizzati considerando il modo in cui i componenti software sono posizionati al loro interno. Un'istanza di un'architettura software, nota anche come architettura di sistema, viene creata quando si prendono delle decisioni riguardanti i componenti del software, come tali componenti interagiscono tra loro e dove tali componenti sono posizionati. Le organizzazioni centralizzate l'argomento si principale di discussione di questa lezione.

Nonostante la mancanza di consenso su molte questioni relative ai sistemi distribuiti, c'è tuttavia un'altra questione su cui la maggior parte dei ricercatori e dei professionisti concorda: pensare ai sistemi distribuiti in termini di client che richiedono servizi dai server è utile per comprendere e gestire la complessità dei sistemi distribuiti. La più rudimentale classificazione tra architetture organizzative di sistemi distribuiti si può ottenere esaminando il numero di livelli presente:

- organizzazioni a livello singolo
- organizzazioni a più livelli.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Architetture client-server

I processi che compongono un sistema distribuito sono separati in due gruppi (potenzialmente sovrapposti) secondo il modello fondamentale client-server. Un processo che implementa un particolare servizio è noto come server. Ad esempio, un servizio di file system o un servizio di database sono entrambi esempi di server. Un processo che effettua una richiesta per un servizio da un server trasmettendo prima la richiesta al server e quindi attendendo che il server risponda è noto come client. Questa interazione client-server, nota anche come comportamento richiesta-risposta (request-response), è illustrata sotto forma di grafico di sequenza dei messaggi nella Figura 1.

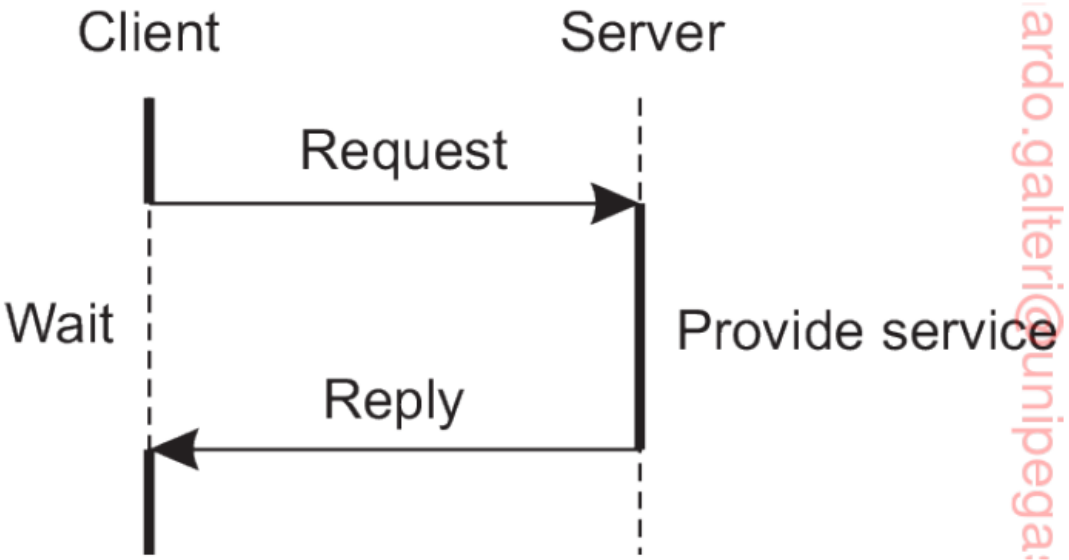


Figura 1 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

Quando la rete sottostante è abbastanza affidabile, come in molte reti locali, la comunicazione tra un client e un server può essere implementata mediante un semplice protocollo senza connessione, rendendo lo stesso protocollo più facile da implementare. Quando un client effettua una richiesta per un servizio in queste circostanze, incapsula semplicemente un messaggio per il server, in cui identifica il servizio che desidera e fornisce tutti i dati di input necessari. Successivamente, il messaggio viene inoltrato al server. Quest'ultimo, a sua volta, attenderà sempre una richiesta in arrivo; quindi, la elaborerà e infine impacchetterà i risultati in un messaggio di risposta che verrà quindi inviato al client. In questo modo, la comunicazione con il client sarà effettiva.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

L'utilizzo di un protocollo che non richiede connessione comporta l'ovvio vantaggio di una maggiore efficienza. Il protocollo di richiesta e risposta appena delineato funziona correttamente finché i messaggi non vengono danneggiati o scomparsi. Non è un compito semplice rendere il protocollo robusto agli errori di trasmissione occasionali che possono verificarsi. Quando non viene ricevuto alcun messaggio di risposta, l'unica cosa che siamo in grado di fare è eventualmente consentire al client di inviare nuovamente la richiesta. Il client, d'altra parte, non è in grado di determinare se il messaggio di richiesta originale è andato perso o se la trasmissione della risposta non ha avuto successo. Questo a tutti gli effetti rappresenta un problema. Se la risposta alla richiesta fosse stata persa nella rete, il nuovo invio della richiesta potrebbe comportare l'esecuzione dell'operazione due volte. Se l'operazione fosse stata qualcosa sulla falsariga di "trasferire 10.000 euro dal mio conto corrente", ad esempio, allora è ovvio che sarebbe stato preferibile per noi segnalare semplicemente un errore. D'altra parte, se l'operazione fosse "dimmi quanti soldi mi sono rimasti", sarebbe perfettamente accettabile inviare nuovamente la richiesta al destinatario. Quando un processo può essere eseguito un numero illimitato di volte senza causare effetti negativi, ci riferiamo a quel processo come idempotente. Dato che alcune richieste sono idempotenti mentre altre no, dovrebbe essere pienamente chiaro che non esiste una soluzione univoca che possa essere applicata al problema dei messaggi persi.

Un protocollo orientato alla connessione affidabile è utilizzato come alternativa da un certo numero di sistemi client-server oggi. A causa delle sue prestazioni relativamente scarse, questa soluzione non è ideale per l'uso in una rete locale; tuttavia, è perfettamente adatto per l'uso in sistemi di ampia area, che sono caratterizzati da una comunicazione intrinsecamente inaffidabile. Le connessioni TCP/IP, ad esempio, sono alla base di quasi tutti i protocolli applicativi utilizzati su Internet. In questo scenario, prima che un client possa inviare una richiesta per un servizio, deve stabilire una connessione al server per inviare la richiesta. Il server utilizzerà in genere la stessa connessione per inviare il messaggio di risposta; una volta inviato il messaggio, la connessione verrà interrotta. Il problema potrebbe essere che stabilire e interrompere una connessione è un processo relativamente costoso, in particolare in situazioni in cui i messaggi di richiesta e risposta sono di dimensioni modeste.

Nel corso della sua esistenza, il modello client-server è stato al centro di molte discussioni e contese. Una delle sfide più importanti è stata determinare come distinguere tra un client e un server in modo significativo. Non dovrebbe sorprendere che non ci sia sempre una netta differenza. Ad esempio, un server per un database distribuito può fungere incessantemente da client a causa del fatto che inoltra le richieste a vari file server responsabili dell'implementazione delle tabelle di database. In questo caso, il server di database gestisce autonomamente tutta l'elaborazione delle query.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Architetture multilivello

La differenziazione in tre livelli logici suggerisce una serie di potenziali modi in cui un'applicazione client-server può essere fisicamente distribuita su più computer. Il modo più semplice per organizzare le cose è usare solo due tipi distinti di macchine:

1. Una macchina client che comprende esclusivamente i programmi responsabili dell'implementazione dell'interfaccia utente.
2. Una macchina server che contiene tutto il resto, ovvero consiste nei programmi che implementano l'elaborazione e il livello dei dati.

In questo tipo di organizzazione, tutto è curato dal server e il client non è altro che un terminale semplice, possibilmente con solo un'interfaccia utente grafica user-friendly. Tuttavia, ci sono un gran numero di altre possibilità. Il livello dell'interfaccia utente, il livello di elaborazione e il livello dei dati sono le tre categorie logiche utilizzate nell'organizzazione della maggior parte delle applicazioni distribuite.

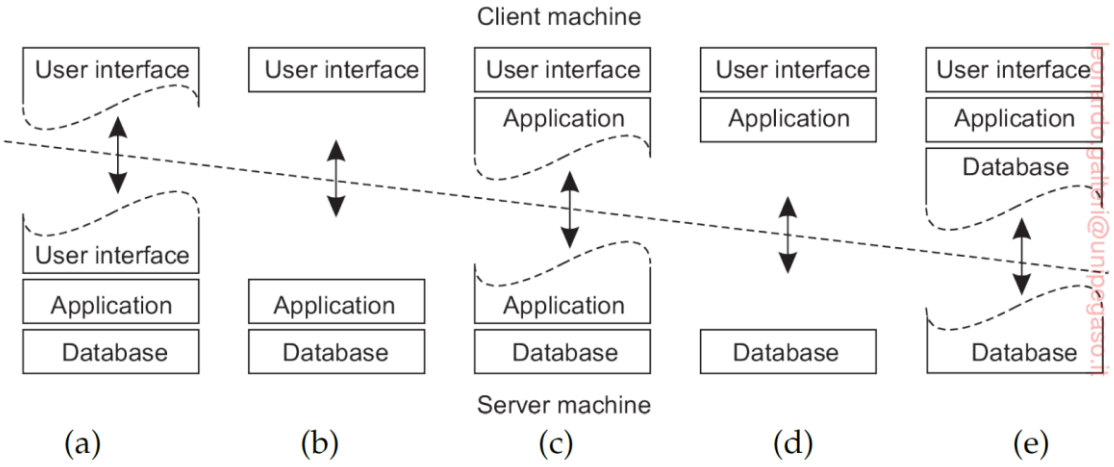


Figura 2 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

Come si può vedere nella Figura 2, una strategia per organizzare client e server è quella di dividere prima questi livelli e poi distribuirli su più macchine. Un'architettura client-server è il risultato del nostro primo passo, che consiste nel differenziare solo tra due tipi di computer: computer client e computer server. Ciò si traduce in quella che è anche comunemente nota come architettura a due livelli.

Una possibile organizzazione consiste nell'avere solo la parte dell'interfaccia utente dipendente dal terminale della macchina client, come mostrato nella Figura 2(a), e dare alle applicazioni il controllo remoto sulla presentazione dei loro dati. Un'altra possibile organizzazione consiste nell'avere entrambe le parti

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

dell'interfaccia utente sul computer client. Un'ulteriore scelta consiste nell'installare l'intero software dell'interfaccia utente sul computer client locale, come illustrato nella Figura 2(b). In situazioni come queste, in genere separiamo l'applicazione in un front-end grafico e un back-end che risiede sul server. Le due parti dell'applicazione comunicano tra loro utilizzando un protocollo univoco per l'applicazione. In questa architettura, il front-end, che comprende il software client, non è responsabile di alcun trattamento diverso da quello richiesto per presentare l'interfaccia dell'applicazione.

Seguendo questa linea di pensiero, un'altra possibilità è che sposteremo alcune delle funzionalità dell'applicazione nel front-end, come illustrato nella Figura 2(c). Uno scenario in cui ciò ha perfettamente senso è quello in cui la domanda utilizza un modulo che, per essere elaborato, deve avere tutti i campi obbligatori completamente compilati. Il front-end è quindi in grado di verificare che il modulo sia stato compilato correttamente e coerentemente e, se necessario, può interagire con l'utente. Un ulteriore esempio dell'organizzazione mostrata nella figura 2(c) è quello di un elaboratore di testi. In questo esempio, le funzioni di editing fondamentali dell'elaboratore di testi vengono eseguite sul lato client, dove operano su dati che sono stati memorizzati localmente nella cache o che sono conservati in memoria. Sul lato server, tuttavia, vengono eseguiti strumenti di supporto più avanzati come il controllo ortografico e grammaticale.

Particolarmente comuni in molte impostazioni client-server sono le configurazioni organizzative illustrate rispettivamente nella Figura 2(d) e 2(e). Questi tipi di organizzazioni vengono utilizzati in situazioni in cui la macchina client è un personal computer (PC) o una workstation collegata tramite una rete a un database o a un file system distribuito. Il computer client è responsabile dell'esecuzione della maggior parte dell'applicazione, ma il server è quello che gestisce tutte le operazioni che coinvolgono file o voci di database. Ad esempio, molte applicazioni bancarie sono progettate per essere eseguite sul dispositivo dell'utente finale, che è anche il luogo in cui l'utente completa varie attività relative alle transazioni. L'applicazione effettua quindi una connessione al database memorizzato sul server della banca e carica le transazioni completate in modo che possano essere elaborate ulteriormente. Lo scenario illustrato nella Figura 2(e) è quello in cui una parte dei dati viene memorizzata sul disco locale del client. Ad esempio, quando un utente naviga in Internet, il computer di un client può gradualmente creare una grande cache sul suo disco rigido contenente le pagine Web visualizzate più di recente.

Quando facciamo la distinzione tra macchine client e macchine server, come abbiamo fatto finora, trascuriamo il punto che un server potrebbe occasionalmente dover agire come client, come mostrato nella Figura 3.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

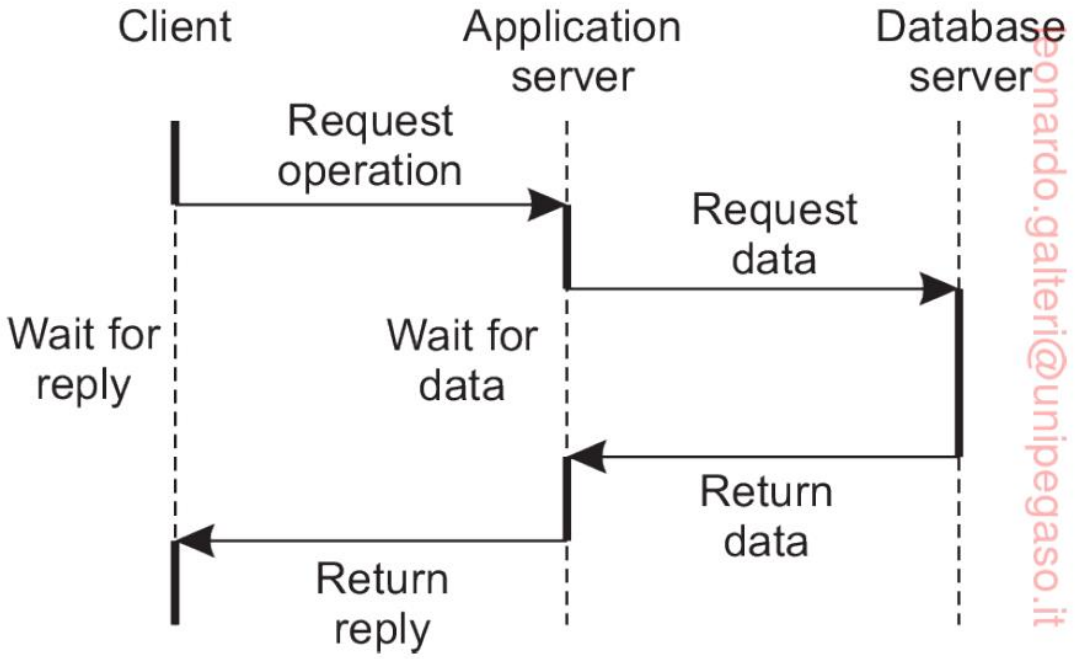


Figura 3 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

Ciò si traduce in un'architettura (fisicamente) a tre livelli. I programmi che sono tradizionalmente considerati parte del livello di elaborazione di questa architettura sono quelli che vengono eseguiti da un server separato. Tuttavia, questi programmi possono anche essere parzialmente distribuiti tra le macchine client e server. L'elaborazione delle transazioni è un buon esempio di un'applicazione tipica che utilizza un'architettura a tre livelli. Un processo separato conosciuto come monitoraggio dell'elaborazione delle transazioni è responsabile del coordinamento di tutte le transazioni che avvengono tra i vari server di dati.

Un'ulteriore istanza di un'architettura a tre livelli, sebbene molto diversa dalla precedente, può essere trovata frequentemente nell'organizzazione dei siti Web. Un server Web funge da punto di ingresso a un sito in questo scenario, passando le richieste a un server applicazioni, in cui avviene l'elaborazione effettiva. Questo server applicazioni comunica con un server di database. Ad esempio, potrebbe essere compito di un server di applicazioni eseguire il codice necessario per controllare lo stock di vari articoli venduti da una libreria online. A tale scopo, potrebbe essere necessaria l'interazione con un database che archivia i dati di inventario non elaborati.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- M. van Steen and A.S. Tanenbaum, Distributed Systems, 3rd ed., distributed-systems.net, 2017.
- Bass L., Clements P., and Kazman R. Software Architecture in Practice. Addison-Wesley, Reading, MA., 2nd edition, 2003.
- Saltzer J. and Kaashoek M. Principles of Computer System Design, An Introduction. Morgan Kaufman, 2009
- Umar A. Object-Oriented Client/Server Internet Environments. Prentice Hall, Upper Saddle River, 1997

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).